MODULE

11

Iterators & generators

Process a 10 GB log file with 10 MB of memory. Every interview asks about yield.

YOU WILL LEARN

How a for loop really works	01_iteration_protocol.py
Generators & yield	02_generators.py
Lazy pipelines	03_pipelines.py
itertools	04_itertools.py
Demo: a 200,000-row CSV	05_demo_big_csv.py
send() & async generators	06_send_and_async_generators.py
Interview Q&A	INTERVIEW.md

WHY GENERATORS

Big data, small memory

THE PROBLEM

You must sum the prices in a **10 GB** CSV. **f.readlines()** loads it all into RAM and the server crashes.

You only ever need **one line at a time**.

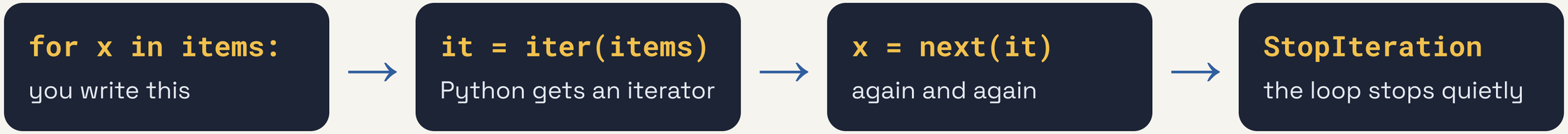
THE IDEA

A generator produces values one by one, on demand, and forgets each value after you've used it.

PICTURE IT ITERATION PROTOCOL

What a for loop really does

🎟️ Think of a **ticket dispenser**. **iter()** gives you the dispenser; each **next()** hands out one ticket; when it's empty it says **StopIteration** and the loop ends.



TRY IT ITERATION PROTOCOL

Do the loop by hand

```
items = ["a", "b"]
it = iter(items)
print(next(it))
print(next(it))
try:
    next(it)
except StopIteration:
    print("StopIteration: the for loop would end here")

print(iter(items) is iter(items))   # a new iterator each time
print(iter(it) is it)               # an iterator returns itself
```

OUTPUT

```
a
b
StopIteration: the for loop would end here
False
True
```

1. An **iterable** (list, str, dict, file) can give you an iterator.
2. An **iterator** remembers its position and has **__next__**.
3. Iterators are **single-use**.

TRY IT GENERATORS

yield pauses the function

```
def countdown(n):
    print("  (start)")
    while n > 0:
        yield n                 # hand out n, then PAUSE here
        n -= 1
    print("  (done)")

gen = countdown(3)
print(gen)                      # nothing has run yet!
for x in gen:
    print("got", x)
```

OUTPUT

```
<generator object countdown at
0x000001DAEEB2CD00>
  (start)
got 3
got 2
got 1
  (done)
```

1. Calling the function only **creates** a generator.
2. Each **next()** runs to the next **yield**, then pauses.
3. Local variables survive between pauses.

STEP BY STEP GENERATORS

Memory: list vs generator

	LIST [X*2 FOR X IN DATA]	GENERATOR (X*2 FOR X IN DATA)
Computed	**all** values immediately	one value per **next()**
Memory for 10 million items	~80 MB+	~200 bytes
Use it twice?	yes	no: it's empty after one pass
len(), indexing [3]	yes	no
Best for	small data you reuse	big data, streams, pipelines

sum(x * 2 for x in data): no extra brackets needed when it's the only argument.

PICTURE IT LAZY PIPELINES

Chain generators like an assembly line

🏭 Each station takes items from the previous one, does **one** job, and passes them on. Only **one item** is on the belt at a time, so a huge file uses almost no memory.



TRY IT LAZY PIPELINES

A log pipeline in 10 lines

```
LOG = ["GET /home 200 12", "GET /api 500 340", "POST /pay 503 900",
       "GET /shop 200 30"]                    # imagine 10 GB

def parse(lines):
    for line in lines:
        method, path, status, ms = line.split()
        yield {"path": path, "status": int(status), "ms": int(ms)}

def errors(rows):
    return (r for r in rows if r["status"] >= 500)

bad = list(errors(parse(LOG)))
print([r["path"] for r in bad])
print("slowest:", max(r["ms"] for r in parse(LOG)), "ms")
```

OUTPUT

```
['/api', '/pay']
slowest: 900 ms
```

1. Each step is small, testable and named.
2. Nothing runs until something **asks** for values (list, max, for).
3. With a real file, **for line in open(path)** is already lazy.

COMMON MISTAKE GENERATORS

Using a generator twice

✗ WRONG

```
squares = (n * n for n in range(5))
print(sum(squares))    # 30
print(sum(squares))    # 0  ???
```

A generator is **used up** after one pass. The second sum gets nothing and silently returns 0.

✓ RIGHT

```
def squares():
    return (n * n for n in range(5))

print(sum(squares()))  # 30
print(sum(squares()))  # 30
```

Need it twice? Create a new generator each time, or store a **list** if the data is small.

TRY IT ITERTOOLS

Ready-made lazy tools

```
from itertools import batched, chain, groupby, islice, count

print(list(islice(count(100), 3)))          # endless, take 3
print(list(chain([1, 2], (3, 4))))          # glue iterables
print(list(batched(range(7), 3)))           # chunks (3.12+)

rows = [("de", "lamp"), ("de", "cable"), ("in", "plug")]
for country, group in groupby(rows, key=lambda r: r[0]):
    print(country, [item for _, item in group])
```

OUTPUT

```
[100, 101, 102]
[1, 2, 3, 4]
[(0, 1, 2), (3, 4, 5), (6,)]
de ['lamp', 'cable']
in ['plug']
```

1. **islice**: take a slice of any iterator.
2. **batched**: send 1,000 rows per DB insert.
3. **groupby** groups **neighbours**: sort by the key first.

IN REAL JOBS GENERATORS

Where they show up at work

Stream a big CSV export

```
def rows():
    for order in Order.objects.iterator():
        yield [order.id, order.total]
```

FastAPI streaming response

```
return StreamingResponse(rows_as_csv(),
                         media_type="text/csv")
```

Paginate an external API

```
def all_users(client):
    page = 1
    while batch := client.get(page):
        yield from batch; page += 1
```

pytest fixtures with cleanup

```
@pytest.fixture
def db():
    conn = connect(); yield conn; conn.close()
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What is a generator, and when do you use one?”

- A function with **yield**. Calling it returns a generator object; values are produced **lazily**, one per **next()**.
- Its state (local variables, position) is kept between calls.
- Use it for **large or infinite data**, streaming and pipelines: memory stays constant.
- A **generator expression (x for x in xs)** is the one-line version.
- It’s single-use, has no len(), and raises StopIteration when finished.

```
def read_big(path):
    with open(path) as f:
        for line in f:
            yield line.rstrip()

total = sum(len(l) for l in read_big("app.log"))
```

Follow-up they ask next: ”What’s the difference between an iterable and an iterator?” (an iterable can create iterators via __iter__; an iterator has __next__ and remembers its position).

INTERVIEW SAY THIS IN THE INTERVIEW

“What does yield from do? What about yield in pytest?”

- **yield from other** hands out every value of another iterable, as if you looped and yielded each one.
- It’s how you **delegate** to a sub-generator (e.g. walking a tree recursively).
- In a pytest fixture or a **@contextmanager**, code **before** yield is setup and code **after** yield is teardown.

```
def walk(tree):
    for name, child in tree.items():
        if isinstance(child, dict):
            yield from walk(child)
        else:
            yield name
```

Follow-up they ask next: ”How would you read a 10 GB file in Python?” (iterate the file object line by line or in chunks; never read() it all).

RECAP REMEMBER THESE

Module 11 in 6 lines

1. **for** = **iter()** + **next()** until **StopIteration**.
2. **yield** pauses a function and keeps its state.
3. Generators are **lazy**: constant memory, but single-use.
4. Chain small generators into a **pipeline**.
5. **itertools**: islice, chain, batched, groupby (sort first!).
6. **yield from** delegates; **yield** in fixtures = setup / teardown.

PRACTICE NOW

Do exercises/ex11_generators.py: build a lazy pipeline over a big log.